

INO Security Audit VI

Two of the round’s three headline fixes hold under tracing. The commits that delivered them also introduced two new ways to destroy user data — one of them inside the function that was just repaired.

DATE	REVISION	SINCE LAST PASS	TEST SUITE
6 September 2026	ac25cc3	2 commits	578 / 578 pass

6/10

OVERALL · UNCHANGED

Genuine, verified progress on the two items that were blocking a Play submission, and the strongest single fix in the series so far — raw Aadhaar and PAN are out of plaintext preferences, **with** an upgrade migration that actually deletes the legacy key. Set against that: **account deletion is broken for the fifth consecutive round**, and the new “forgot passphrase” flow can wipe a vault with no warning at all.

Gains and regressions cancel. The score holds at 6.

TRAJECTORY 4 → 5 → 5 → 6 → 6 → 6

Scores by area

Each area was re-verified from source at `ac25cc3`. Every Critical and High below was confirmed by hand before it was included.

AREA	SCORE	TRAJECTORY	WHAT MOVED IT
Backend & database	5 -1	4 5 5 6 6 5	Account deletion regressed inside the function that was just fixed.
Auth & session	7 —	5 7 6 7 7 7	Push consent genuinely closed; four items untouched for a sixth round.
Crypto & data at rest	5 -1	5 4 5 5 6 5	PII off plaintext prefs, but the passphrase reset now destroys vaults silently.

AREA	SCORE	TRAJECTORY	WHAT MOVED IT
Document sharing	5 —	3 7 5 6 5 5	Proxy secret fixed properly; six other findings untouched, three new minor defects.
Platform & build	6 +1	6 7 5 6 5 6	CI signing is real now and the manifest verifies clean. iOS received nothing.
Privacy & Play policy	7 +1	3 5 4 5 6 7	Both Play blockers closed; a working web deletion flow appeared unprompted.

Open Critical findings

Three. Two were introduced by this round’s own commits.

- CRITICAL
- NEW REGRESSION
- 5TH ROUND BROKEN

Account deletion fails on a column that does not exist

The repaired `delete_account()` deletes expenses by the wrong column name.
The version it replaced had it right.

```
supabase/migrations/20260906020000_phase7_critical_remediations.sql:205
DELETE FROM public.expenses WHERE user_id = v_uid;

supabase/migrations/20260724000000_notes_expenses.sql:52-54
create table if not exists public.expenses (
  id          uuid primary key default gen_random_uuid(),
  auth_user_id  uuid not null default auth.uid()

supabase/migrations/20260904020000_delete_account_rpc.sql:44    (previous version)
DELETE FROM public.expenses WHERE auth_user_id = v_uid;
```

No migration ever adds a `user_id` column to `expenses` — the only later alteration adds `direction`. Every sibling delete in the same function correctly uses `auth_user_id`; this one line is the outlier.

IMPACT

The function raises `column "user_id" does not exist`, the transaction rolls back, and nothing is deleted. Both deletion paths fail together, because the new web flow calls this same RPC. Google Play requires account deletion to work, and the in-app screen correctly reports the failure rather than hiding it — so this is user-visible.

CRITICAL**NEW THIS ROUND**

“Forgot passphrase” can destroy every stored password with no warning

The new destroy-and-start-over flow is real and its confirmation dialog is well written. But the guard deciding whether to *show* that dialog treats “I could not read the vault” as “the vault is empty.”

```
lib/screens/passwords/vault_passphrase_sheet.dart:175-182
} catch (_) {
  serverCount = null;          // count query failed
}
...
final hasEncryptedData = (serverCount != null && serverCount > 0) ||
  (localCount > 0 && hasSealedLocal);
```

A null count with no local copy falls through to the empty-vault branch: biometric prompt, then `resetPassphrase()` replaces the `vault_keys` salt. The destroy dialog never appears. Commit `cf725a2` **deleted** the offline-warning dialog that used to cover exactly this case.

A second route needs no network failure at all. Reads of `w_password_vault` require verified MFA, while `vault_keys` has no such gate on select, insert or delete:

```
supabase/migrations/20260906010000_round7_security_remediation.sql:45-47
CREATE POLICY "w_password_vault: owner reads own" ...
  USING (auth.uid() = auth_user_id AND public.ino_is_aal2_satisfied());

supabase/migrations/20260727140000_vault_keys.sql:49-61
create policy "vault_keys: owner reads own" on public.vault_keys
  for select using (auth_user_id = auth.uid());          -- no aal2 gate
```

At AAL1 the count query returns *zero rows, not an error* — row-level security filters them out silently — so the vault reads as empty and the re-key still succeeds.

IMPACT

The unlock sheet appears every time the app returns to the foreground, not only when a passphrase was genuinely forgotten. A user who knows their passphrase, taps the button once on a fresh install with a weak connection, and approves a biometric prompt labelled “reset vault passphrase” loses every stored password permanently, having been shown no warning. This is the same failure shape as previous rounds: the fix is present, the guard is drawn around the wrong condition.

CRITICAL**3RD ROUND UNFIXED**

Decrypted documents are rendered by a script fetched from a public CDN

```
share-frontend/app/s/[token]/DocViewer.tsx:13
share-frontend/app/v/[token]/OneTimeDoc.tsx:13
  pdfjs.GlobalWorkerOptions.workerSrc =
    `https://unpkg.com/pdfjs-dist@${pdfjs.version}/build/pdf.worker.min.mjs`;

share-frontend/next.config.js:32
  script-src 'self' 'unsafe-inline' 'unsafe-eval' unpkg.com;
```

IMPACT

The worker receives every decrypted PDF byte the viewer renders. If unpkg or any of its CDN edges serves a modified worker, that content can be exfiltrated. Web workers cannot carry subresource integrity, so no integrity check is available — the only fix is to serve the worker from the app's own origin and drop `unpkg.com` from the policy.

Open High findings

Fifteen verified. Grouped by where they live.

HIGH**NEW THIS ROUND**

`destroyVault()` reports success when the server delete failed

`lib/services/password_store.dart:99-119` — the remote delete sits in an inner `catch` that only logs. Local state is wiped and `true` returned regardless, so the sheet proceeds to re-key. Offline, or at AAL1 where the delete matches zero rows without throwing, the old ciphertext returns on the next sync and cannot be decrypted under the new key — permanent unreadable entries the user was told were destroyed.

HIGH**NEW THIS ROUND**

The error message on the destroy path is an untranslated key

`vault_passphrase_sheet.dart:145` uses `vaultPassphraseSetupFailed`, which exists in none of the three language tables — only `vaultSetupFailed` and `vaultResetFailed` do. A user whose re-key fails immediately after destroying their vault sees the literal string `vaultPassphraseSetupFailed`. The parity test passes because the key is uniformly absent from all three locales.

HIGH**6TH ROUND UNFIXED**

The MFA challenge screen can be dismissed with a back gesture

`lib/screens/auth/mfa_challenge_screen.dart` has no `PopScope`; ten other screens in the app do. On the account-switch path the screen is *pushed* onto the previous account's live stack (`auth_flow.dart:63–72`), so a back swipe pops it and leaves an AAL1 session running on the shell. This chains directly into the second route of the vault-destruction Critical above.

HIGH**CARRIED**

Sign-out is local-scope, so removed accounts keep a valid refresh token

`auth_service.dart:397` calls `signOut()` with no scope, and gotrue 2.26.0 defaults to `SignOutScope.local`. The comments in `account_switcher.dart:66–71` and `:173–175` both assert the saved token “is about to be revoked” by that call. It is not. A token captured before removal stays valid server-side until natural expiry.

HIGH**3RD ROUND UNFIXED**

Share tokens carry 48 bits of entropy

```
supabase/migrations/20260707000000_share_tokens.sql:17
default substr(replace(gen_random_uuid()::text, '-', ''), 1, 12)
```

Twelve hex characters, against an endpoint whose only limiter is 60 requests per minute per IP. Three validators disagree on the format: Dart accepts `{3,128}` (`share_link_validator.dart:45`) under a docstring claiming “16 to 128”, the edge function accepts `{6,32}`. Meanwhile `ShareCodecService.generateToken` generates 192-bit tokens and is referenced only by a test — a decoy that makes the entropy look four times better than it is.

HIGH**3RD ROUND UNFIXED**

loadCards lists documents without checking ownership

```
supabase/functions/share/index.ts:675-690
.from("documents")
.select("id, name, category, file_path")    // no auth_user_id
.in("id", share.document_ids);

supabase/functions/share/index.ts:818-829  (serveFile, correct)
if (doc.auth_user_id !== share.owner_id) { ... reject ... }
```

Any document id present in `document_ids` has its name and category returned, whoever owns it. Only the byte fetch is guarded. The asymmetry between these two functions is unchanged across three audits.

HIGH**CARRIED**

Unlock tokens are signed with the service-role key and survive password removal

`index.ts:289-331` — the HMAC secret is the service-role key itself, and the payload is only `{shareId, exp}`. Removing or changing a share password does not invalidate outstanding tokens; a recipient who unlocked at T reads files until T+1h regardless. Mixing the password hash or a per-share counter into the payload fixes the second half.

HIGH**CARRIED**

Received documents are written to temp storage and never deleted

Three sites in `shared_documents_screen.dart` (`:170` , `:195` , `:231`), plus `vault_detail_screen.dart:180,189` and `view_once_share_screen.dart:143` .

Every document ever received via a share link accumulates in plaintext in the cache directory, outliving the share's expiry and revocation. The correct pattern — delete in a `finally` block — already exists in a sibling file at `view_once_viewer_screen.dart:260-264` .

HIGH**NEW THIS ROUND**

OTP verification for account deletion has no rate limit

```
share-frontend/app/api/account/delete-request/route.ts:14-15,59
const MAX_PER_IP = 5;  const MAX_PER_EMAIL = 3;
if (!checkRateLimit(ipRateMap, ip, MAX_PER_IP) || ...) return 429;

share-frontend/app/api/account/delete-confirm/route.ts
(no rate limiting of any kind)
```

The request half is carefully limited on both IP and email. The confirm half, which verifies a six-digit code and then irreversibly deletes the account, has no bucket at all. Only GoTrue's own limits stand between a brute-force and someone else's data. Mirror the buckets from `delete-request` .

HIGH**CARRIED**

Data export covers 3 of about 15 domains, and masks what it does include

`data_export_service.dart:68-112` emits `account` , `settings` and `documents` only — no expenses, notes, reminders, cards, wallets, password vault, family vaults, sessions or consents. It also masks `record_number` (`:99`) and strips the extraction envelope (`:102`). An export that redacts the user's own Aadhaar number back to them is not portability. `PRIVACY_POLICY.md:87` still promises the "entire document library."

HIGH**CARRIED**

The privacy policy omits about 13 collected data categories

Section 3 lists seven; card last-four, expenses, net-worth aggregates, reminder content, UPI QR data, share analytics, Firebase installation ID, notification content, voice audio, IP address, offline copies, MFA factors, session records and consent records are all absent. IP addresses are not mentioned anywhere in the document. This section is the direct input to the Play Data Safety form — fix it before filling that form in.

HIGH**CARRIED**

Retention is a three-way contradiction and no purge job is ever scheduled

```
supabase/migrations/20260813000000_push_log.sql:121      interval '180 days'
supabase/migrations/20260904030001_harden_notifications_privacy.sql:52 interval '30 days'
PRIVACY_POLICY.md:79      "pruned after 90 days"
```

The published figure matches neither. More importantly, `cron.schedule` appears in no executed migration — `purge_push_log()`, `purge_notification_outbox()`, `purge_sent_notifications()` and `purge_expired_view_once_shares()` are all defined, permissioned, and never called. The word “automatically” in the published policy is false as deployed.

HIGH**CARRIED**

The 18+ declaration is English-only and stored where the user can rewrite it

`signup_screen.dart:395` hardcodes the attestation sentence; `google_terms_consent_screen.dart` is unlocalized throughout. Hindi and Telugu users sign an English legal declaration pointing at in-app Terms that contain no age clause in any locale. The attestation is recorded in Supabase `user_metadata` via `auth.updateUser()` (`auth_service.dart:82-93`) — client-writable, so not a defensible audit record.

HIGH**CARRIED**

Recipient IP addresses are stored raw on three paths, undisclosed, and never purged

The RPC path now hashes (`phase7:431,460`) but *unsalted* — an unsalted SHA-256 of an IPv4 is a 2^{32} brute force, so it stays personal data. Three edge-function paths still write or match the raw value (`index.ts:229-244` , `:200-203` , `:270`).

`vault_invite_audit_logs` stores raw invitee email and raw client IP for people who never used the app. Nothing purges `share_rate_limits` , and `delete_account()` does not touch it.

HIGH**CARRIED**

iOS: no development team is set, so universal links cannot ship

`ios/Runner.xcodeproj/project.pbxproj` contains zero `DEVELOPMENT_TEAM` entries. All three Runner configurations reference `Runner/Runner.entitlements` but carry neither a team nor a signing style, so the `associated-domains` entitlement cannot be provisioned from a clean checkout or in CI. The Podfile also still compiles out microphone and speech (`ios/Podfile:65-66`) while `Info.plist:81-84` advertises both.

Verified fixed this round

Each traced end to end rather than accepted from the diff.

- ✓ **Proxy secret fails closed on both sides.** The hardcoded literal is purged repo-wide; an unset or empty variable, and an empty token, are all correctly rejected. This was the previous pass's headline Critical.
- ✓ **Play blocker — legal pages are indexable.** Privacy, terms and delete-account each override the site-wide `noindex` , and the two alias routes inherit it correctly through live ES re-exports.
- ✓ **Play blocker — the five-way host contradiction is gone.** Every load-bearing artifact now says `share.inoapp.com` , confirmed in the merged manifest.
- ✓ **A working web deletion flow appeared.** Email → OTP → confirm, enumeration-safe, fail-closed on missing credentials, invoking the RPC under the verified user's own token. Undermined only by the broken RPC itself.

- ✓ **Raw Aadhaar, PAN and the extraction envelope are off plaintext preferences** — with a real upgrade migration on the hot path that deletes the legacy key. The strongest fix in the series.
- ✓ **Push consent is no longer fabricated**, gated both at the call site and inside the service, and cold-start re-registration is genuinely inert.
- ✓ **CI signing is real**. The keystore is decoded and all four properties Gradle reads are written; the path resolves correctly against the app module. All six actions are SHA-pinned.
- ✓ **Backup exclusions enumerate every domain** in both cloud-backup and device-transfer, and the attribute survives manifest merging.
- ✓ **The forgot-passphrase dead end is gone** — the flow is reachable and completes. See the Critical above for what it does when it gets there.

Build and test verification

Run against the working tree, not inferred from source.

TEST SUITE 578 passed0 failed	STATIC ANALYSIS 1 infounnecessary_import	GRADLE CONFIGURE exit 0help · offline
MANIFEST MERGE exit 0processDebugMainManifest	MERGED PERMISSIONS 135 media strips held	APP-LINK HOSTS 1share.inoapp.com

One caveat on the suite: several of the new tests assert that a string appears somewhere in a source file rather than testing behaviour. The backup-rules test would pass with an empty `<device-transfer>` block — the exact bug it was written for — and the proxy-secret test reimplements the TypeScript in Dart, then pins the defective fallback line in place with an assertion.

`live_verification_round6_test.dart` and `password_vault_reseal_corruption_test.dart` are the counter-examples: real state, real assertions.

What needs doing outside the code

Ordered by leverage. The first item blocks the submission.

01 Register inoapp.com and point share.inoapp.com at Vercel

The app now ships share links on this host, so until DNS resolves, every QR code the app generates is dead *and* the Play Console privacy-policy URL returns 404. Add the domain under Vercel → Settings → Domains and CNAME to `cname.vercel-dns.com`.

02 Register inoapp.in and create three mailboxes

`privacy@`, `support@` and `delete-account@`. Play verifies the deletion contact. Note this is a second registrable domain — the web host is `.com`, every contact address is `.in`. Consolidating onto one would be simpler.

03 Set the Vercel environment variables

`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_FUNCTIONS_URL` and `SHARE_PROXY_SECRET`. The deletion routes return 503 without the anon key, and the proxy secret must match the Supabase secret exactly or IP forwarding silently disables.

04 Schedule the purge jobs in the Supabase SQL editor

Enable `pg_cron`, then schedule `purge_push_log()`, `purge_notification_outbox()`, `purge_sent_notifications()` and `purge_expired_view_once_shares()`. All four exist and none is ever called; the published policy claims they run automatically.

05 Do not fill the Data Safety form from the current policy

It will be inaccurate on IP address, voice audio, notification content and installation identifiers. Correct section 3 first, then fill the form from it. An inaccurate Data Safety declaration is a post-approval enforcement risk, not a pre-approval one.

06 Confirm the deep-link files serve correctly once DNS exists

`assetlinks.json` and both Apple site-association paths must return `application/json` over HTTPS with no redirect. Then check Google's verification status on a device.

07 Have the entity details reviewed

`PRIVACY_POLICY.md:15` names a company and a Bengaluru address. If that entity is not registered, the DPDP data-fiduciary declaration is itself a misrepresentation.

Method: four domain-scoped review passes over revision `ac25cc3`, followed by hand-verification of every Critical and High against the source before inclusion. Findings that did not survive that check were dropped.

Not verifiable from this host: iOS toolchain behaviour, live DNS and header responses, device-level clipboard and temp-file behaviour, and whether Supabase edge functions populate `cf-connecting-ip` in production.

